Algorithms and data structures

DB4002
31 March 2026

HÖGSKOLAN I HALMSTAD

Assignment Package 2 – Lab1

General instructions: How and when to submit and what to submit are explained on the submission page on Blackboard. This document describes the assignments.

This information is available as part of the course description available in Blackboard:

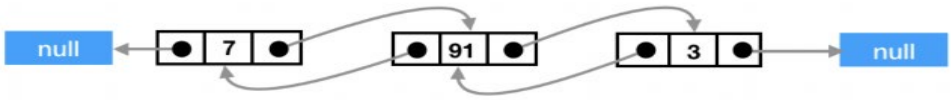
- All assignment packages are divided into two parts called **Part 1** and **Part 2**. Part 1 is done to get a pass on the exam, while Part 2 is done to get bonus points for a higher grade (4, 5) on the second part of the exam (see the information below about the exam structure). That's it, Part 1 must be done to get a pass, while Part 2 is only done to collect bonus points for a higher grade on the exam.
- The total number of bonus points you can get by solving the tasks on Part 2 for each assignment package is **10**. That' it, you can collect half of the points for a higher grade on the exam by solving these Part 2 tasks. The bonus points you get for the exam are equal to the sum of the bonus points you have received from the solutions to the Part 2 tasks.

Algorithms and data structures

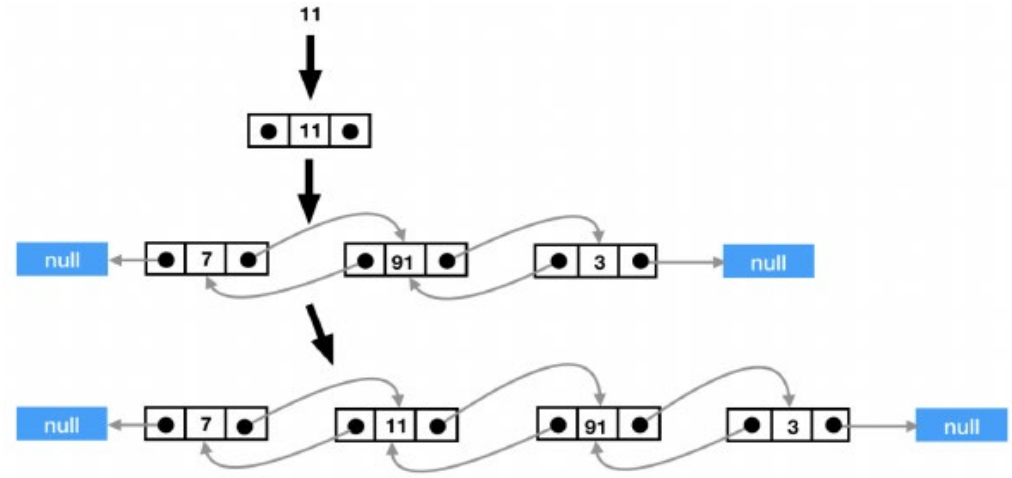
DB4002
31 March 2026

Assignments - part 1

1. In this task you will implement a **doubly linked list**. A doubly linked list consists of so-called nodes that both contain a value and also point to other nodes in a chain. This pointing to other nodes is done using two variables that are references to two other nodes. One is a reference to the node that comes before in the chain (i.e. in front of the list) and the other is a reference to the node that comes after in the chain (i.e. after in the list). Below we have an example of a doubly linked list consisting of three nodes.



In such a doubly linked list, you add data as shown in the illustration below. You put the value in a node, then you specify where the node should be in the list by assigning the node references for the node and updating the node references for the two nodes where you squeezed in between.



And you have the API for the data type:
public class DoublyLinkedList<T> **implements** Iterable<T>{ ... }

```
// Create an empty list.
DoublyLinkedList()

// Add t to the end of the list.
void add(T t)

// Add t at location index in the list.
void add(int index, T t)

// Returns the value at location index.
T get(int index)

// Returns the first value in the list.
T getFirst()

// Returns the last value in the list.
T getLast()
```

```
/*
If the value t exists, all occurrences of t are removed.
The number of occurrences of t removed is returned.
If nothing is removed, 0 is returned.
*/
int remove(T t)

// Removes the value at position index and returns that value.
T remove(int index)

// Removes and returns the last value in the list.
T removeLast()

// Removes and returns the first value in the list.
T removeFirst()

// Checks if the list is empty.
boolean isEmpty()

// Returns the size of the list.
int size()

// Removes all elements in the list. Makes it empty.
void clear()

// Converts the list to a string.
String toString()

// Returns an iterator for the list.
Iterator<T> iterator()
```

a) (1 point): Start by creating a generic data type for a node (e.g. a class private class ListNode{//...} inside the class DoublyLinkedList). This class contains three instance variables that represent the value (of type T) and references to a preceding and a succeeding node (of type **ListNode**) in the chain.

b) (14 points): Now you will use these nodes to create a generic doubly linked list according to the specification (**API**) above. The specified methods **must be** implemented in this task, but other (additional) methods may be implemented if you wish.

Algorithms and data structures

DB4002
31 March 2026

HÖGSKOLAN I HALMSTAD

2. Write a program ReverseDLL where you use the data structure Stack that was implemented in the lectures to reverse the order of a doubly linked list. What you are going to do is the following.

a) (1 point): Program a method
public static <T> **void** reverse(DoublyLinkedList<T> dll)

In this method you create a stack, then you go through all the elements in the list and in each iteration you put the element on the stack. Then you empty the list. Then you pick the elements off the stack and put the elements **in order** in the list until the stack is empty.

b) (1 point): In main you are going to test that reverse works. One way to do this for an arbitrary list is by reversing the list **twice** and verifying that you get the same list as you started with.
- Therefore, in main you are going to read the value from the command line and **create two doubly linked lists** with the same elements.
- You are going to reverse one of the lists twice.
- Then, you should go through the list you didn't reverse and the list you reversed twice to test that they are the same.
- The program **should print either Reverse worked** for the list followed by the list you entered, **or Reverse did not work** for the list followed by both lists.
- Use the program with no command line arguments, with one, with two, with nine, and with ten values on the command line.

3. a) (1 point): In this task you will extend **DoublyLinkedList** with another instance method:
public void addAtFirstSmaller(T t, Comparator<T> cmp)
- In this method you go through the elements in the list backwards (i.e. starting from the end of the list and working your way forward).
- As soon as an element in the list is smaller than t, t is added to the list after that element.
- You can say that you go backwards through the list and find the first value in the list that is smaller than t (first smaller) and add t after the element (creating a node that you enclose in parentheses).
- Smaller and larger are defined by using the **compare** method of **cmp**.
- If all the elements in the list are larger than t, t is added as the first element in the list.

b) (1 point): Now use the method **addAtFirstSmaller**(T t, Comparator<T> cmp) above to implement Insertion Sort to sort an array **a**. The implementation is simple with the method above.

You are going to write a program **SortDLL** where you first program a method
public static <T> **void** dllInsertionSort(T[] a, Comparator<T> cmp)

According to these steps:
Step 1: Create an empty list of type DoublyLinkedList. We can call it dll.
Step 2: Iterate over the array **a**. For each element **t** in **a**, add **t** to dll using **addAtFirstSmaller**.
Step 3: Iterate over dll and add each element to the corresponding location in **a**.

(c) (1 point): In main, you will use the techniques we showed in the TestGenericSort class to test that the algorithm sorts and you will compare the execution time of dllInsertionSort and GenericSorting.insertionSort

Algorithms and data structures

DB4002
31 March 2026

HÖGSKOLAN I HALMSTAD

BONUS!!!

Submission Assignments - Part 2

General rules:

- Your code must be **generic** where the task says so, and must use **Comparable** or a **Comparator** exactly as requested.
- For timing/experiments, use **System.nanoTime()** or **System.currentTimeMillis()** and run each test **multiple times** (warm-up + average), otherwise results will look random.
- Unless stated, you may use **Arrays.sort** as the **O(n log n)** sort (the point is the algorithmic idea around it, not re-implementing sorting again).

4. Iterative Quicksort.

What you must implement: A iterative Quicksort for an array **a** of elements that implement Comparable.

a) (1 point) Iterative Quicksort using a stack.

- You must not use recursion.
- Use a stack that stores index pairs (**lo, hi**) for subarrays.
- Start by pushing (0, n-1).
- While the stack is not empty:
  - Pop a range (lo, hi)
  - Partition it
  - Push the two resulting subranges (if they are non-trivial)

Important detail (stack size control): Push the larger subrange first and the smaller second. This keeps the maximum stack depth around O(log n) in practice.

b) (1 point) Median-of-3 pivot. Instead of always picking the first element as pivot:

- Choose pivot as the median of three elements: a[lo], a[mid], a[hi]
- Swap that median element into a[lo] before partitioning (so your partition code stays simple).

You do not need median-of-5.

c) (1 point) Cutoff to insertion sort: Add a cutoff threshold C (choose e.g. C = 32):

- If a subrange length <= C, do not partition it.
- Just run insertion sort on that subrange.

You do not need to compare many cutoff values. Just implement one sensible cutoff and use it.

d) (1 point) Small experiment: Compare running time for:

- Random arrays
- Already sorted arrays

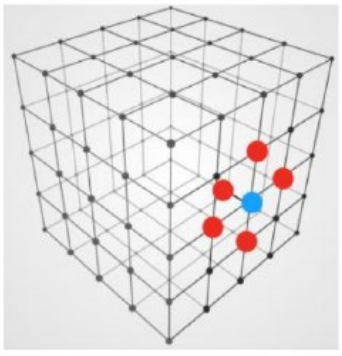
You do not need “nearly sorted” as a third category.
What to report:

- A small table or short text with times for 2–3 input sizes (e.g. 10^4, 2*10^4, 4*10^4)
- One or two sentences: “what changed and why”.

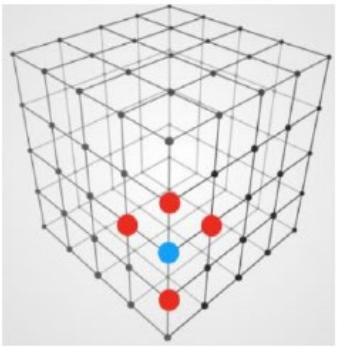
Algorithms and data structures

DB4002
31 March 2026

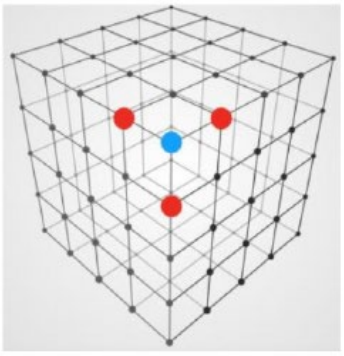
5. Local minimum in a 3D array in O(n^2)



Side point: 5 neighbors



Edge point: 4 neighbors



Corner point: 3 neighbors

You are given an 3D array int[][][] a of size n x n x n.

A local minimum is a cell whose value is <= **all of its neighbors**.

Neighbors:

- For interior points: up to 6 neighbors (±1 step in each axis)
- On faces/edges/corners: fewer neighbors (only those inside bounds)

a) (6 points) Write an algorithm that finds one local minimum in worst-case **O(n^2).**

Tips:
A direct scan of all n^3 cells is too slow. You need to reduce the search space.

One standard approach (acceptable here):

1. Pick the middle “slice” along one dimension, e.g. k = n/2 (a 2D plane of size n x n)
2. Find the minimum element on that plane in O(n^2)
3. Compare it to its neighbors “in front” and “behind” the plane (the k-1 and k+1 neighbors, if they exist)
4. If the plane-minimum is not a local minimum, move to the half that contains a smaller neighbor and repeat on that half.

This gives a recurrence like: $$T(n) = T(n/2) + O(n^2)$$ So overall O(n^2).